Laboration 6 - Asynkron programmering och APIer

Saknad

10/18/2026

Försök 1

Pågår

NÄSTA: Lämna in uppgift

Lägg till kommentar

Unlimited Attempts Allowed

11/1/2026

Information

Syfte/mål

Syftet med uppgiften är att:

- Förstå hur asynkron programmering används när data hämtas från externa källor.
- Använda `fetch()` för att göra anrop till ett externt REST-API.
- Använda `async/await` för att hantera asynkrona operationer.
- Hantera och använda JSON-data som returneras från ett API.
- Hantera fel som kan uppstå vid externa anrop.
- Bearbeta hämtad data och presentera den dynamiskt i DOM.
- Använda tidigare kunskaper om formulär, event, funktioner, arrayer, objekt och DOM-manipulation.
- Strukturera JavaScript-kod i mindre funktioner med tydliga ansvarsområden.
- Använda Git och GitHub för att versionshantera programkod.

I laborationen bygger du vidare på dina tidigare kunskaper i JavaScript och DOM-manipulation. Nu ska dessa användas tillsammans med asynkron programmering för att hämta och presentera data från en extern webbtjänst.

Teori och läsanvisningar

För att förbereda dig inför uppgiften, ta del av föreläsningar, teori och läsanvisningar samt eventuella övningar och quizar i modulens avsnitt på kurswebbplatsen. Prova också att experimentera med kodexempel i din utvecklingsmiljö för att stärka din förståelse innan du sätter igång med uppgiften.

Prova gärna att köra de kodexempel som finns i teorin själv och experimentera med att ändra värden och kod. Att skriva och testa små mängder kod åt gången är en viktig del av att lära sig programmering.

Du kommer även att använda formulär, event, DOM-manipulation, arrayer, objekt, loopar och funktioner från tidigare moment.

Uppgiftsbeskrivning

I denna laboration ska du skapa funktionaliteten till en receptsökare.

Du får färdiga HTML- och CSS-filer som utgör gränssnittet till applikationen. Din uppgift är att skriva JavaScript-koden som gör det möjligt för användaren att söka efter recept.

Receptdata ska hämtas från det externa APIet: **DummyJSON Recipes (https://dummyjson.com/docs/recipes)** . När användaren skriver en sökfras och skickar formuläret ska ett API-anrop göras där sökfrasen används i anropet.

Det första receptet i sökresultatet ska sedan presenteras på webbsidan.

Applikationen ska kunna:

1. läsa den sökfras som användaren skriver i formuläret
2. kontrollera att en sökfras har angetts
3. göra ett asynkront API-anrop med `fetch()`
4. använda `async/await` för att invänta och hantera svaret
5. omvandla API-svaret från JSON
6. kontrollera om sökningen gav något resultat
7. presentera det första receptet från sökresultatet
8. skapa receptets innehåll dynamiskt med JavaScript
9. hantera fel som kan uppstå vid API-anropet
10. ersätta tidigare recept när användaren gör en ny sökning

Demonstration

Nedan finns en film (*utan ljud*) som visar exempel på hur en färdig lösning kan fungera.

Arbetsfiler

Till laborationen finns **färdiga arbetsfiler (https://canvas.miun.se/courses/11718/files/831099?wrap=1)** . Ladda ner dessa och packa upp filerna på valfri plats på din dator.

Projektet innehåller bland annat:

```
index.html
css/
    styles.css
js/
    main.js
```

All programmering i laborationen ska göras i `main.js`. Du ska inte ändra HTML- eller CSS-filerna.

HTML- och CSS-filerna innehåller det färdiga gränssnittet och den formatering som ska användas för applikationen.

I HTML-filen finns även ett exempel på den HTML-struktur som ska skapas med JavaScript när ett recept skrivs ut. Använd denna struktur när du skapar receptets element.

Versionshantering med Git och GitHub

När du har laddat ner och packat upp arbetsfilerna ska du skapa ett **eget Git-repository** för laborationen och publicera detta på GitHub.

Du ska använda Git för att versionshantera projektet under **hela arbetet med laborationen** och göra regelbundna commits allt eftersom du utvecklar funktionaliteten.

Versionshistoriken ska tydligt visa hur lösningen har utvecklats över tid. Det är alltså inte tillräckligt att skapa en eller ett fåtal commits först när laborationen är färdig.

Skriv commit-meddelanden som kortfattat beskriver **vad du har gjort eller förändrat i koden**, exempelvis:

```
Lade till validering av sökformulär
Implementerade hämtning av recept från API
Lade till utskrift av receptinformation
Skapade listor för ingredienser och instruktioner
Lade till hantering av fel vid API-anrop
```

Undvik generella commit-meddelanden som `ändringar`, `uppgift`, `fix` eller `klart`.

Sökning efter recept

Applikationen innehåller ett formulär där användaren kan ange en sökfras.

När formuläret skickas ska JavaScript läsa värdet från sökfältet och använda det för att göra ett anrop mot **DummyJSON (https://dummyjson.com/docs/recipes)** :s API för recept.

Följande endpoint ska användas:

```
https://dummyjson.com/recipes/search?q=SÖKFRAS
```

Exempelvis ska en sökning efter `pasta` resultera i ett anrop till:

```
https://dummyjson.com/recipes/search?q=pasta
```

Anropet ska göras med `fetch()` och hanteras med `async/await`.

Svaret från API:t ska konverteras från JSON till ett JavaScript-objekt innan informationen används.

Validering av sökfältet

Innan ett API-anrop görs ska du kontrollera att användaren har angett en sökfras.

Om sökfältet är tomt ska:

- ett tydligt felmeddelande visas på sidan
- inget API-anrop göras

Felmeddelande kan skrivas ut till p-elementet med id `error`.

Eventuella tidigare felmeddelanden ska tas bort när användaren gör en ny giltig sökning.

Hantera API-svaret

API:t returnerar ett objekt som bland annat innehåller en array med recept.

I denna laboration ska du endast visa **det första receptet i sökresultatet**.

Om sökningen inte ger några recept ska ett tydligt meddelande visas för användaren, exempelvis att inget recept kunde hittas och att en annan sökfras kan provas.
Meddelandet kan skrivas ut till section-elementet med id `message`.

Om ett recept hittas ska eventuell tidigare receptinformation tas bort innan det nya receptet visas.

Visa receptet

Receptet ska skapas och läggas till i DOM med JavaScript.

HTML-filen innehåller ett kommenterat exempel på vilken struktur det skapade receptet ska ha.

För receptet ska följande information från API:t visas:

- receptets namn
- bild
- antal portioner (`servings`)
- förberedelsetid (`prepTimeMinutes`)
- tillagningstid (`cookTimeMinutes`)
- svårighetsgrad (`difficulty`)
- ingredienser (`ingredients`)
- instruktioner (`instructions`)

Ingredienserna ska skrivas ut som en **punktlista (`ul`)**.

Instruktionerna ska skrivas ut som en **numrerad lista (`ol`)**.

Eftersom innehållet i recepten hämtas från en extern tjänst kommer receptnamn, ingredienser och instruktioner att vara på engelska. Även rubriker och information som hör till själva receptet kan därför skrivas på engelska enligt exempelstrukturen i HTML-filen.

När användaren gör en ny sökning ska det tidigare receptet tas bort och ersättas med resultatet från den nya sökningen.

Felhantering vid API-anrop

Ett anrop till en extern webbtjänst kan misslyckas, exempelvis på grund av problem med nätverket eller tjänsten som används.

API-anropet ska därför hanteras med `try/catch`.

Du ska även kontrollera att svaret från servern är giltigt innan du försöker använda den returnerade datan.

Om något går fel vid hämtningen ska:

- eventuellt tidigare recept tas bort
- ett tydligt felmeddelande visas för användaren

Felmeddelandet kan skrivas ut till section-elementet med id `message`.

Teknisk information om felet kan även skrivas ut i webbläsarens konsol för att underlätta felsökning.

Strukturera programmet

Dela upp programmet i mindre funktioner med tydliga ansvarsområden.

Programmet behöver exempelvis funktionalitet för att:

- validera sökformuläret
- hämta recept från API:t
- skriva ut ett recept i DOM

Använd lämpliga eventlyssnare för att hantera formuläret och koppla samman programmets olika delar.

Försök låta varje funktion utföra en tydlig uppgift och återanvänd befintliga funktioner där det är lämpligt.

Din programkod ska vara **väl kommenterad och läsbar**. Kommentarerna ska hjälpa till att förklara vad olika delar av programmet gör. Använd även tydliga och beskrivande namn på variabler och funktioner.

Tips och ledtrådar

Arbeta med **en del i taget och testa ofta**.

Ett lämpligt arbetssätt kan exempelvis vara att:

1. börja med formuläret och kontrollera att du kan läsa den angivna sökfrasen samt lägg till validering.
2. testa API-anropet och skriv först ut resultatet med `console.log()`
3. undersök objektet som returneras och identifiera den information du behöver
4. skriv ut grundläggande information om receptet i DOM
5. skapa listorna med ingredienser och instruktioner
6. lägg slutligen till validering och felhantering för APIet.

Krav för godkänt

Laborationen bedöms med **Godkänd (G)** eller **Underkänd med möjlighet till komplettering (Fx)**.

För att laborationen ska anses vara godkänd krävs att:

- Applikationen är färdigställd och fungerar enligt uppgiftsbeskrivningen ovan.
- Receptdata hämtas från angiven webbtjänst med `fetch()` och `async/await` utifrån användarens sökfras.
- Hämtad JSON-data bearbetas och receptet presenteras dynamiskt i DOM enligt angiven struktur.
- Validering och felhantering fungerar och tydliga meddelanden visas när en sökning inte kan genomföras eller inte ger något resultat.
- JavaScript-koden är väl kommenterad, läsbar och strukturerad i lämpliga funktioner med tydliga ansvarsområden.
- Git har använts under hela arbetet och repositoryts versionshistorik innehåller **regelbundna commits med tydliga och beskrivande commit-meddelanden**.

Publicering

När laborationen är färdig ska webbplatsen publiceras på en publikt tillgänglig webbhost.

Använd någon av de publiceringsmetoder eller plattformar som du arbetat med tidigare i kursen.

Testa alltid den **publicerade versionen** innan du lämnar in laborationen och kontrollera bland annat att:

- sökningen fungerar
- ett recept hämtas och visas korrekt
- ingredienser och instruktioner visas
- ett nytt recept ersätter det tidigare
- valideringen av sökfältet fungerar
- ett tydligt meddelande visas om inget recept hittas
- fel vid API-anrop hanteras

Inlämning

Vid inlämning ska du lämna:

- en länk till den publicerade webbplatsen
- en länk till ditt GitHub-repository

Kontrollera att båda länkarna fungerar innan du skickar in laborationen.

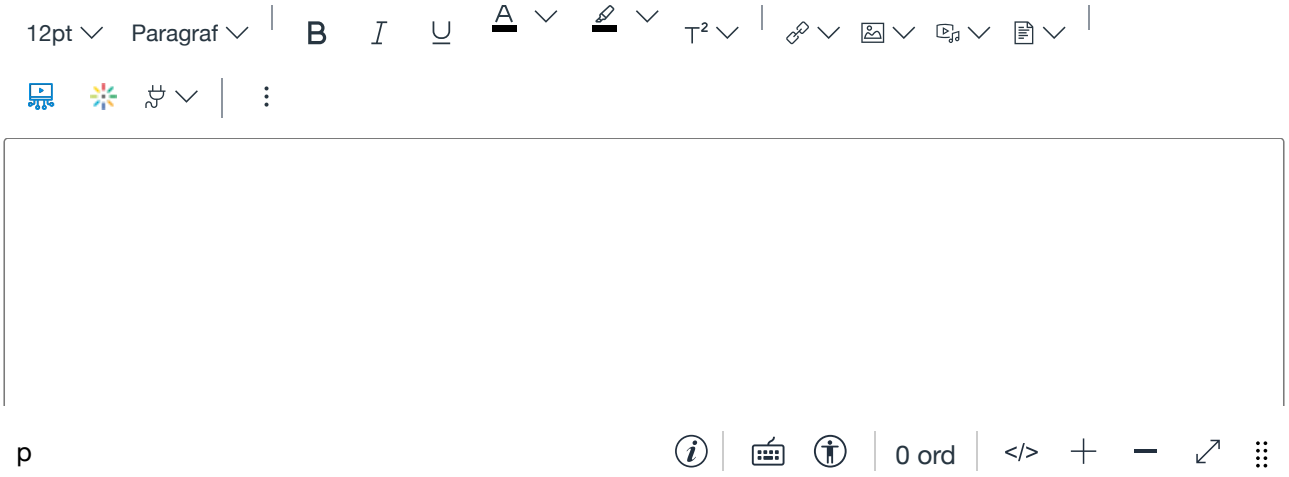
Repositoryt ska innehålla samtliga filer för laborationen samt en versionshistorik som visar att Git har använts regelbundet under arbetets gång.

Kom ihåg att göra GitHub-repot publikt under rättningstiden.

Visa matris

Moment 6			
Kriterier	**Bedömningar**		**Poäng**
Receptsökaren fungerar enligt instruktionerna och hämtar samt visar recept från API:t.	**Uppfylld** **1 poäng**	**Ej uppfylld** **0 poäng**	/1 poäng
fetch() och async/await används för att hämta och bearbeta data från API:t.	**Uppfylld** **1 poäng**	**Ej uppfylld** **0 poäng**	/1 poäng
Sökningen valideras och fel samt sökningar utan resultat hanteras med tydliga meddelanden.	**Uppfylld** **1 poäng**	**Ej uppfylld** **0 poäng**	/1 poäng
JavaScript-koden är läsbar, kommenterad och uppdelad i funktioner med tydliga ansvarsområden.	**Uppfylld** **1 poäng**	**Ej uppfylld** **0 poäng**	/1 poäng
GitHub-repot visar en tydlig utvecklingsprocess med regelbundna och beskrivande commits.	**Uppfylld** **1 poäng**	**Ej uppfylld** **0 poäng**	/1 poäng

Redigera Visa Infoga Format Verktyg Tabell



(https://canvas.miun.se/courses/11718/modules/items/231321)

(https://canvas.miun.se/courses/11718/modules/items/230